DOCUMENTATION

MiniCFL — Project Documentation

> Full documentation for the **MiniCFL** Azure AZ-104 project.
> This file consolidates the working notes in this Obsidian vault into one reference. The detailed source notes live under `Project Notes/`.

Table of Contents

---

1. Overview

MiniCFL is a small Azure project built for the **AZ-104 (Microsoft Certified: Azure Administrator Associate)** course. The visible product is a simple web page that simulates **one CFL train** travelling between **Petange** and **Luxembourg** and shows where the train currently is.

The train itself is intentionally simple. Its real purpose is to provide a concrete reason to **design, deploy, and administer a complete Azure environment** the way an

Azure administrator would — covering identity, governance, compute, networking, and monitoring.

> **Core principle (from `Project Index.md`):** keep MiniCFL simple — one train simulation, one web interface, one container, one VM database, one Azure Function timer, with **real Azure administration evidence** around it.

---

2. Project Goals

Prove understanding of the main AZ-104 areas by building a small but complete environment:

- **Identity & access** — Microsoft Entra ID, groups, and RBAC.
- **Governance** — management groups, subscriptions, resource groups, tags, policy, budgets.
- **Compute** — App Service, one container, one VM, and one Azure Function.
- **Networking** — VNet, subnets, NSGs, and a Load Balancer demo.
- **Monitoring** — Azure Monitor, logs, alerts, and cost checks.

Objectives & Key Results (OKR)

- A running CFL simulator.
- Demonstrated AZ-104 skills.

KPIs

KPI	Target
Deployment time	From zero to *X* minutes
Azure knowledge	From 25% → 60%
Cost reduction	From 100% → 60%

Milestones

1. User management
2. Resource creation
3. Cost and monitoring
4. Software

---

3. Requirements

Course / assignment requirements

- Use **Teams and Planner** for communication and planning.
- Create a **Web App Service**.
- Use **one container**.
- Use **one Virtual Machine** that runs the database.
- Create and explain the **network**.
- Assess **how roles are given and how access works**.
- Create **imaginary workers**.
- Use a **Load Balancer**.
- **Document** what was created and collect evidence.

How MiniCFL satisfies each requirement

Requirement	How MiniCFL shows it
Teams and Planner	Planning screenshots, tasks, and daily reporting
Web App Service	Host the train UI in App Service / Web App for Containers
One container	Dockerize the web app and push it to ACR
One VM	Run the database on a small Azure VM (`vm-SQL`)
Network	VNet, subnets, NSGs, and connectivity tests
Roles and access	Entra groups and RBAC at resource group scope
Imaginary workers	Test users assigned to groups
Load Balancer	A small load-balancing demo, separate from App Service
Evidence	Screenshots, portal views, logs, and short explanations

> **Scope decision:** the app does **not** need to be complex. A simple web UI is enough if the surrounding Azure resources are well explained.
>
> **Load Balancer note:** Azure Load Balancer is for VMs / VM Scale Sets, **not** a front door for App Service. It is implemented as a **separate networking demo** (two small VMs or a VMSS).

---

4. Architecture

The architecture diagram is exported from LucidChart as:

- `export/MiniCFL.png`
- `export/MiniCFL.svg`

```
User
  |
  v
App Service / Web App for Containers
  |
  v
MiniCFL container image from Azure Container Registry
  |
  v
VNet
  |
  +-- Database VM in database subnet
  |
  +-- Azure Function timer updates train state
  |
  +-- Azure Monitor, Log Analytics, alerts
```

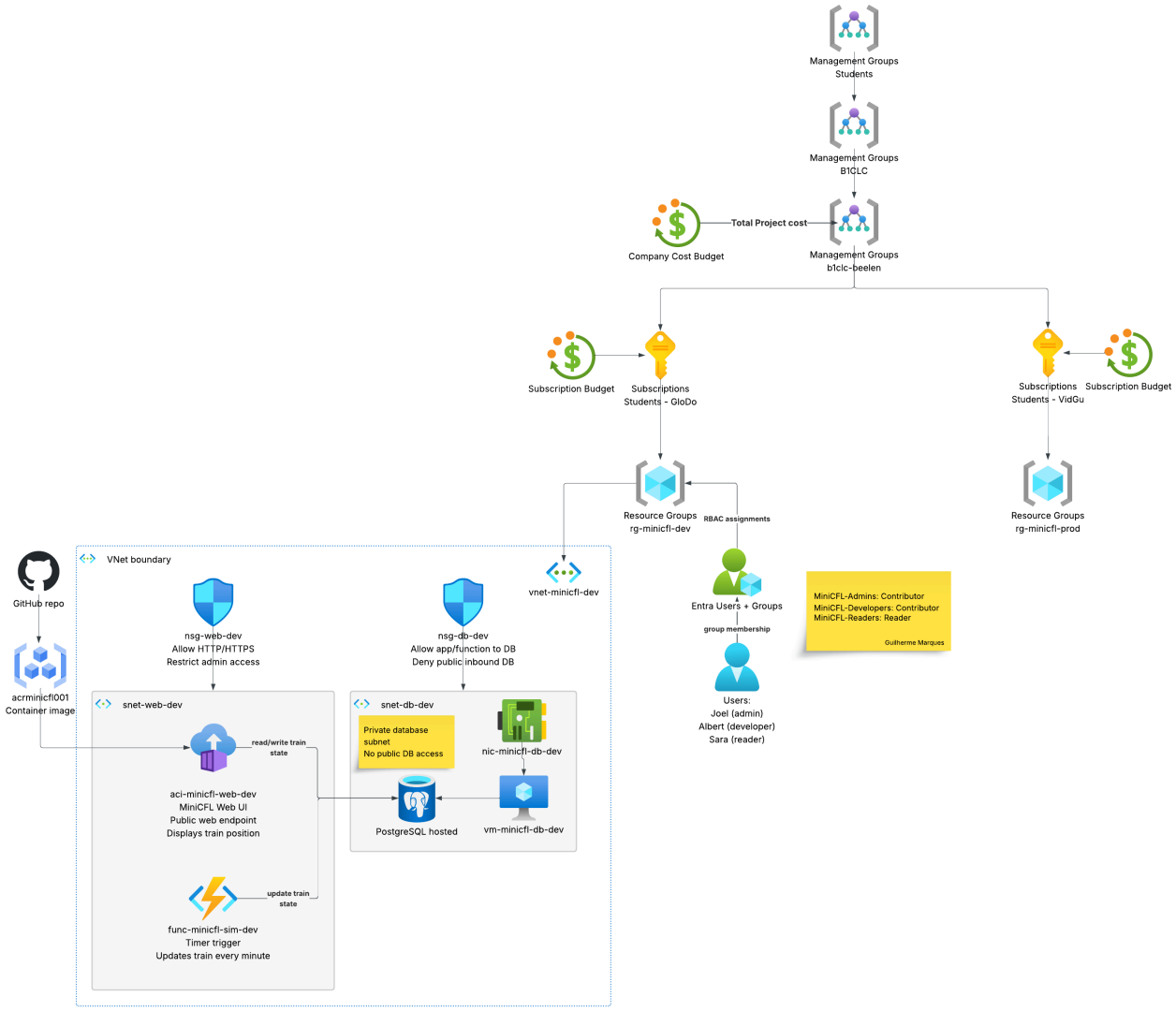
Environments

Environment	Resource group	Subscription	Region
Development	`rg-minicfl-dev`	Azure for Students - GloDo	France Central
Production / demo	`rg-minicfl-prod`	Azure for Students - VidGu	Sweden Central

`rg-minicfl-prod` serves as the controlled demo / release environment for the course.

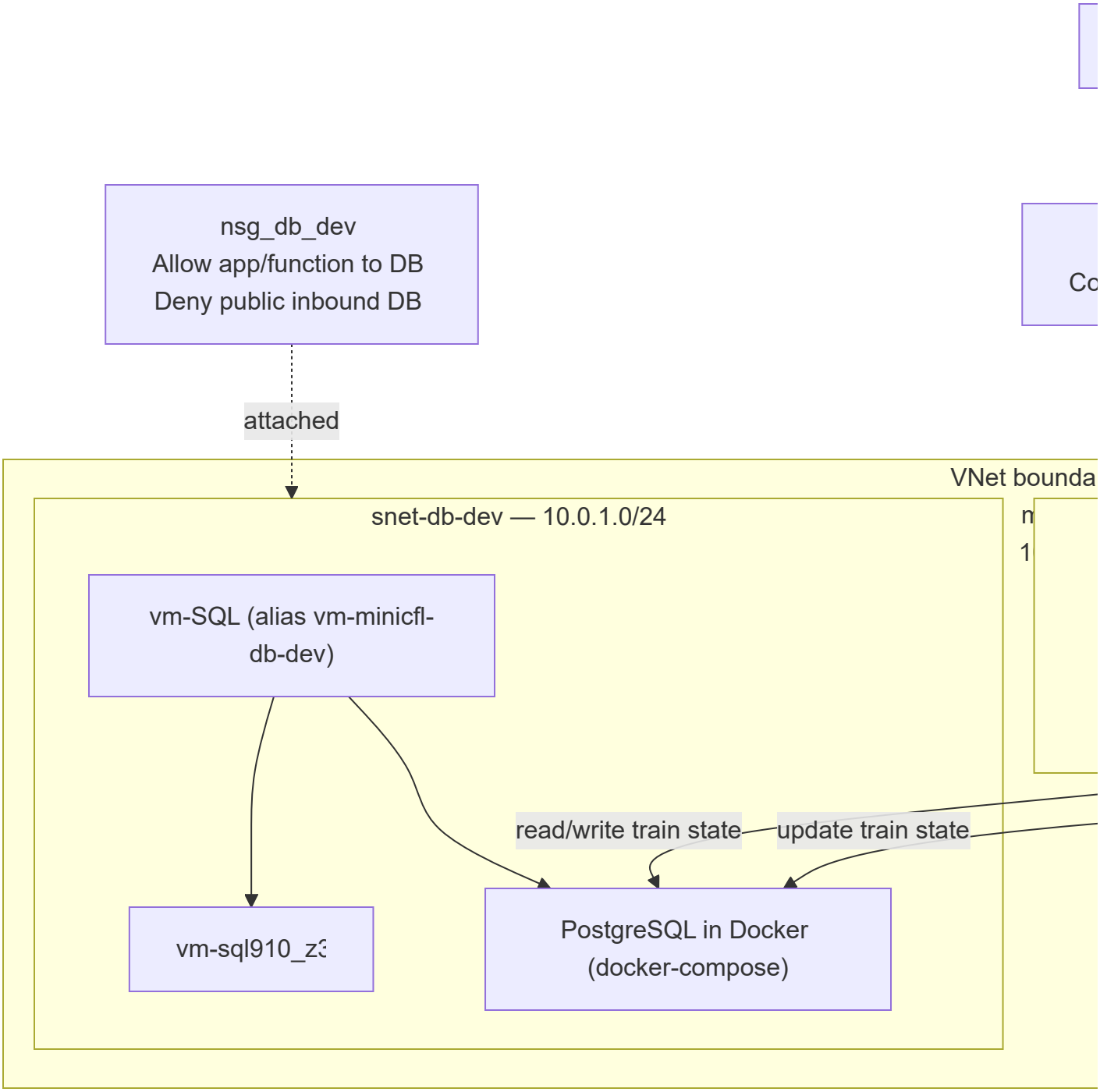
Detailed dev architecture (`rg-minicfl-dev`)

The diagram below is the concrete development build inside resource group `rg-minicfl-dev` and virtual network `vnet-minicfl-dev`.



> **To embed the screenshot:** the architecture image is exported to `export/MiniCFL.png` (and `.svg`). The `![[...]]` line above will render it in Obsidian.

The same architecture as a version-controlled Mermaid diagram:



Components shown:

Component	Name	Role
Source	GitHub repo	App source code
Container registry	`acrminicfldev001`	Stores the MiniCFL container image
Web UI	`app-minicfl-web-dev`	MiniCFL web interface on **Azure App Service**; public

Component	Name	Role
		web endpoint, displays train position; read/writes train state to the DB
App Service plan	`ASP-rgminicfldev-94c1`	Hosts the App Service
Function	`func-minicfl-sim-dev`	Timer trigger that updates the train state every minute
Web subnet	`snet-web-dev`	Hosts the web UI and function (`10.0.0.0/24`)
Web NSG	`nsg-web-dev`	Allow HTTP/HTTPS; restrict admin access; **attached**
Database subnet	`snet-db-dev`	Private database subnet for the PostgreSQL VM (`10.0.1.0/24`)
Database VM	`vm-SQL` (alias `vm-minicfl-db-dev`)	Linux VM running PostgreSQL via **Docker Compose**; private IP `10.0.1.4`
DB NIC	`vm-sql910_z3`	VM network interface in `snet-db-dev`
Public IP	`vm-SQL-ip`	Public IP for restricted SSH administration; observed address `40.66.41.94`
SSH key	`vm-SQL_key`	SSH key resource for VM administration
OS disk	`vm-SQL_OsDisk_1_08ab4bf881a4433d8963411903738d45`	Managed OS disk
Database NSG	`nsg_db_dev`	Allow app/function → DB; deny public inbound; **attached**

Component	Name	Role
Application Insights	`app-minicfl-web-dev`	App performance monitoring for the web app
Log Analytics workspace	`logsanalyticsws-minicfl-dev`	Central log workspace
Smart detector alert rule	`Failure Anomalies - app-minicfl-web-dev`	Azure-created App Service anomaly detection
Action group	`Application Insights Smart Detection`	Azure-created notification target for smart detection

> **Note on compute choice:** the current dev build uses **Azure App Service** for the web UI. Earlier ACI references were part of an older design and should not be used for the current implementation.
>
> **Note on the database:** PostgreSQL runs inside a Docker container on `vm-SQL`, managed with `docker-compose`. The VM also has Docker Compose v1 installed. The `schema.sql` and `seed.sql` are loaded automatically on first container start.

---

5. Azure Organization & Governance

Management group hierarchy

Parent	Child	Type	ID / Notes
Tenant Root Group	Students	Management group	`Students`
Students	Students Pay as you go	Subscription	`4c977b1a-c08d-4e1a-a742-a9cf48b505fd`
Students	B1CLC	Management group	`B1CLC`
B1CLC	b1clc-beelen	Management group	`b1clc-beelen`
b1clc-beelen	Azure for Students - GloDo	Subscription	`31609096-dfdf-40c2-8c20-250b025bd552`

Parent	Child	Type	ID / Notes
b1clc-beelen	Azure for Students - VidGu	Subscription	`190ebd65-61cd-4da9-be23-c301ba189209`

Why this structure matters: management groups organize subscriptions, policy can be assigned above subscriptions, and cost/governance can be reviewed at a higher scope.

Resource groups

Resource group	Subscription	Region	Purpose
`rg-minicfl-dev`	Azure for Students - GloDo	France Central	Development
`rg-minicfl-prod`	Azure for Students - VidGu	Sweden Central	Production / demo
`NetworkWatcherRG`	Azure for Students - GloDo	France Central	Azure-created network monitoring
`NetworkWatcherRG`	Azure for Students - VidGu	Sweden Central	Azure-created network monitoring

Policies to demonstrate

Policy idea	Purpose
Require tag on resources	Governance and cost tracking
Allowed locations	Keep resources in approved regions (France Central, Sweden Central)
Allowed VM SKUs	Prevent expensive VM sizes
Restrict public/admin access on database VM	Allow only required SSH administration; keep PostgreSQL closed to the internet
Require secure transfer on storage	Improve storage security

---

6. Identity & Access

Groups (preferred over per-user assignments)

Group	Purpose
`MiniCFL-Admins`	Manage the project environment
`MiniCFL-Developers`	Deploy and update app resources
`MiniCFL-Testers`	Test and read project resources
`MiniCFL-Readers`	Read-only access for reviewers

Imaginary workers

Worker	Example role	Group
`alex.admin`	Project administrator	`MiniCFL-Admins`
`sam.dev`	Developer	`MiniCFL-Developers`
`nora.tester`	Tester	`MiniCFL-Testers`
`reviewer.reader`	Teacher / reviewer	`MiniCFL-Readers`

Suggested RBAC assignments

Group	Scope	Role	Reason
`MiniCFL-Admins`	`rg-minicfl-dev` and `rg-minicfl-prod`	Contributor	Manage without Owner
`MiniCFL-Developers`	`rg-minicfl-dev`	Contributor	Build and deploy in dev
`MiniCFL-Developers`	`rg-minicfl-prod`	Reader	Inspect prod, no changes
`MiniCFL-Testers`	`rg-minicfl-dev`	Reader	Test and view
`MiniCFL-Readers`	Both project RGs	Reader	View-only evidence access
`MiniCFL-Developers`	Azure Container Registry	AcrPush	Push container images

> **Least privilege:** avoid Owner unless a specific setup task requires it.

Access review questions (for the final report)

- Who can deploy resources? Who can only read?
- Which scope was used: management group, subscription, resource group, or resource?

- Which resources are public vs private?
- Is the database blocked from the internet?
- Which policies reduce mistakes or high costs?

---

7. Naming, Tags & Cost Control

Naming convention

Resource type	Example
Resource group	`rg-minicfl-dev`, `rg-minicfl-prod`
Virtual network	`vnet-minicfl-dev`
Web subnet	`snet-web-dev`
Database subnet	`snet-db-dev`
Network security group	`nsg-web-dev`, `nsg_db_dev`
Virtual machine	`vm-SQL` (alias `vm-minicfl-db-dev`)
Public IP address	`vm-SQL-ip`
Network interface	`vm-sql910_z3`
SSH key	`vm-SQL_key`
Container registry	`acrminicfldev001`
Function app	`func-minicfl-sim-dev`
App Service	`app-minicfl-web-dev`
App Service plan	`ASP-rgminicfldev-94c1`
Storage account	`stminicfl001`
Application Insights	`app-minicfl-web-dev`
Log Analytics workspace	`logsanalyticsws-minicfl-dev`
Load Balancer	`lb-minicfl-dev`

Tags

Tag	Example value	Purpose
`Project`	`MiniCFL`	Identify project resources
`Environment`	`Dev` / `Prod`	Separate environments

Tag	Example value	Purpose
`Owner`	`Beelen`	Show responsibility
`Course`	`AZ-104`	Connect resource to assignment
`CostCenter`	`SchoolProject`	Support cost tracking

Budget

Setting	Value
Budget name	`budget-minicfl`
Period	Monthly
Alert thresholds	30%, 50%, 75%, 100% actual cost; 50% forecasted

Budgets were created on **18/06/2026** on the `b1clc-beelen` management group and per-subscription (`Students - GloDo` and `Students - VidGu`).
Alert thresholds are set at 30%, 50%, 75%, 100% actual cost and 50% forecasted.

---

8. The Train Simulation

Route

```
Petange -> Niederkorn -> Differdange -> Esch-sur-Alzette -> Bettembourg -> Luxembourg
```

The return direction uses the same stops in reverse.

Example 30-minute trip

Order	Station	Planned minute	Description
1	Petange	0	Train starts
2	Niederkorn	5	First stop
3	Differdange	9	Second stop
4	Esch-sur-Alzette	15	Middle of route
5	Bettembourg	23	Last stop before Luxembourg
6	Luxembourg	30	Final station

Operating time

- Luxembourg local time.
- Trains run ~04:00 → 01:00; outside that window status is `stopped_for_night`.

Train state model (database fields)

Field	Example	Purpose
`train_id`	`CFL-001`	Identifies the train
`route_id`	`petange-luxembourg`	Identifies the route
`direction`	`to_luxembourg`	Current direction
`status`	`between_stops`	Current running state
`previous_stop`	`Differdange`	Last stop passed
`next_stop`	`Esch-sur-Alzette`	Next planned stop
`planned_arrival_time`	`08:15`	Timetable arrival
`estimated_arrival_time`	`08:18`	Arrival including delay
`delay_minutes`	`3`	Current delay
`segment_progress_percent`	`50`	Position between two stops
`last_update_time`	`2026-06-08T08:12:00`	Last simulation update

Status values

Status	Meaning
`not_started`	Route not started yet
`waiting_at_station`	At a station
`between_stops`	Travelling between two stations
`arrived`	Reached final station
`late`	Delayed vs planned timetable
`returning`	Going back in the opposite direction
`stopped_for_night`	Outside operating hours

Azure Function logic (runs every minute)

1. Check current Luxembourg time.
2. Check if the train should be operating.

3. Determine direction.
4. Calculate elapsed minutes in the current trip.
5. Add delay minutes.
6. Find previous and next stop.
7. Calculate progress between the two stops.
8. Save the new train state to the database.
9. Write a log entry.

UI requirements

Must show: train lane, station stops, train position, direction, current/previous stop, next stop, planned arrival, estimated arrival, delay badge, last update time.

Status colors: **green** on time · **orange** small delay · **red** large delay · **gray** stopped for night.

```
CFL-001
Petange -> Luxembourg

Current: between Differdange and Esch-sur-Alzette
Next stop: Esch-sur-Alzette
Planned arrival: 08:15
Estimated arrival: 08:18
Status: Late by 3 min
Last update: 08:12
```

---

9. Azure Resources & AZ-104 Mapping

Resource	Purpose	AZ-104 area
Management group `b1clc-beelen`	Organize project subscriptions	Governance
Subscriptions	Billing and resource boundary	Governance
Resource groups	Separate dev and prod resources	Governance
Entra users and groups	Manage people and imaginary workers	Identity
RBAC	Control who can deploy / test / read	Identity & governance

Resource	Purpose	AZ-104 area
Azure Policy	Enforce tags, regions, SKUs, public IPs	Governance
Budget alerts	Control student credit usage	Cost management
Azure Container Registry	Store the app container image	Compute
App Service / Web App for Containers	Run the web interface	Compute (PaaS)
Azure Function	Timer that updates the train state	Compute (FaaS)
Virtual Machine	Database server	Compute (IaaS)
Managed disk	VM OS / data storage	Storage & compute
VNet and subnets	Separate web and database traffic	Networking
NSGs	Allow/deny traffic	Networking & security
Load Balancer	Demonstrate load balancing for VMs	Networking
Azure Monitor	Metrics, logs, dashboards, alerts	Monitoring
Network Watcher	Network troubleshooting & connectivity checks	Monitoring & networking

> **Accuracy notes:** don't claim the Load Balancer protects App Service unless that architecture is actually built. Terraform/Bicep are optional — portal deployment is acceptable if each step is explained. The app stays simple; it is a scenario for AZ-104 skills.

---

10. Implementation Plan

The plan follows the AZ-104 learning order: **understand the concept → build it in Azure → collect evidence.**

Phase	Goal	Key tasks
1. Planning	Set up the work	Teams, Planner, notes; ownership; naming & tags
2. Governance base	Understand the hierarchy	Review MGs, confirm subscriptions/RGs, tags, budgets
3. Identity & access	Controlled access	Create workers, groups, RBAC at RG scope; test Reader

Phase	Goal	Key tasks
4. Network	Network foundation	VNet, subnets, NSGs; block public DB; connectivity tests
5. Database VM	IaaS compute	Small VM in DB subnet, managed disk, Docker, PostgreSQL, seeding, monitoring, backup
6. Container & Web App	Visible app	Build app, Docker image, push to ACR, run on App Service
7. Function timer	Auto-update train	Function App, timer trigger every minute, save to DB, log
8. Load Balancer demo	Satisfy LB requirement	2 VMs / VMSS behind a Load Balancer; show probe + rule
9. Monitoring & report	Operate the environment	Dashboard, metrics, alerts, action group, final report

Detailed per-phase task and evidence lists are in `Project Notes/Implementation Plan.md`.

Recommended build order

1. Base: management groups, subscriptions, resource groups, tags.
2. Identity: users, groups, RBAC.
3. Networking: VNet, subnets, NSGs, access tests.
4. Compute: VM database (Docker + PostgreSQL), App Service, container, Function.
5. Operations: Monitor, logs, alerts, budget, Advisor.
6. Collect evidence for the final report.

---

11. Evidence & Reporting

The final report must **show evidence**, not just state what was done. Track progress in `Project Notes/Evidence Checklist.md`.

Evidence status snapshot

Area	Captured so far
Governance	✅ Management group hierarchy · ✅ Resource groups · ✅ Budgets (MG + subscriptions) · ☐ Subscriptions, tags, policy, Advisor

Area	Captured so far
Identity & access	☐ Users, groups, RBAC, access tests
Compute	✅ VM overview (`vm-SQL`) · ✅ Docker + PostgreSQL running via docker-compose · ✅ Database seeded · ☐ App Service, ACR, Function evidence
Networking	✅ VNet (`vnet-minicfl-dev`) · ✅ Subnets (`snet-web-dev` , `snet-db-dev`) · ✅ NSG rules · ✅ NSG attached to subnets · ☐ LB, Network Watcher
Monitoring	☐ Dashboard, metrics, alerts, backup
Application	✅ Database rows with train state (`CFL-001` seeded) · ☐ Running UI, function logs
Report writing	✅ Architecture diagram (`export/MiniCFL.png`) · ☐ Skill mapping, problems & solutions, conclusion

Report should include

- Architecture diagram (`export/MiniCFL.png`)
- AZ-104 skill mapping table
- Problems and solutions table (see Known Issues & Solutions)
- Personal contribution notes
- Final conclusion

---

12. Team & Project Management

- **Communication:** Microsoft Teams
- **Task board:** Microsoft Planner
- **Notes:** OneNote / Markdown (this vault)
- **Source code:** GitHub (if an app repository is created)

Daily tasks

- Watching Pluralsight videos
- Testing resources
- Writing down completed tasks
- Communicating progress
- Looking for alternatives
- Documenting

Learning material

`Pluralsight Notes/` contains course notes and PDFs for the AZ-104 tracks **"Deploy and Manage Azure Compute Resources"** and **"Manage Azure Identities and Governance"** (App Services, Containers, Virtual Machines).

---

13. Progress Log

Consolidated from the reporting notes.

Date	Activity
01/06/2026	Project announcement; created Teams / Planner / OneNote environment; initial tasks defined. Watched first Pluralsight video.
04/06/2026	Created OKR, KPI, and milestones.
05/06/2026	Watched Pluralsight videos 2 and 3.
13/06/2026	Finished Pluralsight videos and notes (one video remaining).
15/06/2026	Set up management group with team as Contributors; added subscription.
16/06/2026	Created role assignments on resource groups; created user groups (to add users later).
18/06/2026	Created budgets on management group and subscriptions (`b1clc-beelen` → `budget-minicfl` ; GloDo; VidGu → `cfl-dev-budget`) with alert thresholds at 30/50/75/100% actual and 50% forecasted.
19/06/2026	**Networking & Database infrastructure complete (Marios).** Created VNet `vnet-minicfl-dev` with address space `10.0.0.0/16` and 2 subnets (`snet-web-dev` `10.0.0.0/24` , `snet-db-dev` `10.0.1.0/24`). Created and attached NSGs (`nsg-web-dev` , `nsg_db_dev`) with inbound rules for HTTP/HTTPS/SSH and PostgreSQL. Deployed `vm-SQL` (Ubuntu 24.04, Standard_B2ats_v2, private IP `10.0.1.4`) in DB subnet. Installed Docker and Docker Compose on the VM. Deployed PostgreSQL via `docker-compose.db-vm.yml` (image `postgres:16-alpine` , container `minicfl-db-vm`). Configured `.env` with DB credentials and bound to `0.0.0.0:5432` . Verified seeded data: `SELECT * FROM train_state` returns `CFL-001` with route `petange-luxembourg` , status `not_started` . Evidence captured: VNet, subnets, NSGs, VM, Docker, PostgreSQL, and seeded DB screenshots.

---

14. Known Issues & Solutions

Problems encountered during the build and how they were resolved.

Problem	Solution
VNet address space defaulted to `10.0.0.0/16` instead of the originally planned `10.10.0.0/16`	Adjusted subnet ranges to match (`10.0.0.0/24` and `10.0.1.0/24`) and updated documentation
Docker Compose v1 doesn't support `--env-file` flag	Created a `.env` file directly in the `app` folder on the VM and removed the `--env-file` flag from the compose command
Docker Compose v1 doesn't support the `name` field at top of compose file	Removed the `name` line and added `version: "3.8"` to the file for v1 compatibility
Docker container name conflict on restart	Used `docker rm -f minicfl-db-vm` to force-remove the old container before recreating it
`.env` file had extra `EOF'` and `EOF` text from heredoc issue	Overwrote the file cleanly with echo commands
Port initially bound to `127.0.0.1:5432` (localhost only)	Fixed `MINICFL_DB_BIND_HOST` to `0.0.0.0` so VNet resources can reach the database
Admin access from Luxembourg network (CGNAT)	Gave the DB VM a public IP with NSG restriction; documented as "restricted admin access"

---

15. Repository Structure

```
MiniCFL-Az104/
├── DOCUMENTATION.md                    <- this file
├── export/                             <- architecture diagram exports
│   ├── MiniCFL.png
│   └── MiniCFL.svg
├── Project Notes/                      <- current working documentation
│   ├── Project Index.md                <- reading order + main direction
│   ├── Project Description.md
│   ├── Requirements.md
│   ├── Architecture and AZ-104 Mapping.md
│   ├── Governace and acces.md
│   ├── Tools.md
│   ├── Siumulation.md
│   ├── Implementation Plan.md
│   ├── Evidence Checklist.md
│   └── AI recommednation/              <- older AI-generated draft (reference
only)
```

```
├── Project Managment/
│   ├── Project management.md         <- OKR / KPI / milestones
│   └── Jamey JAGER - Project Management Example.md
├── Reporting/
│   ├── Chapter 1.md
│   └── Chapter 1-LeEVO.md
├── Personal Reporting/
│   ├── Guilherme.md
│   ├── Donnny.md
│   └── Marios.md                     <- latest infrastructure progress
└── Pluralsight Notes/                <- AZ-104 course notes + PDFs
    └── Deploy and Manage Azure Compute Resources/
```

> **Reading order for the source notes** (`Project Index.md`): Project Description → Requirements → Architecture and AZ-104 Mapping → Governance and Access → Tools → Simulation → Implementation Plan → Evidence Checklist.
>
> The `Project Notes/AI recommednation/` folder is an **older AI-generated draft** kept for reference only — the current working notes are the files listed directly under `Project Notes/`.